

Dokumentacja testowa

1. Wstęp.....	3
a) Tworzenie nazw plików aplikacji	3
b) Tworzenie nazw plików testowych	3
c) Wprowadzenie.....	3
d) Podejście do testów	3
2. Opis testów	4
a) Testowany obiekt	4
b) Narzędzia	4
c) Funkcjonalność nietestowana	4
3. Plan testów.....	4
a) Opis.....	4
b) Obszar pokrycia testów	5
4. Finalny zakres testów	6
5. Zakres pokrycia testami	6
6. Raport błędów wykrytych w czasie działania aplikacji.....	7
7. Końcowy raport i potwierdzenie zgodności końcowej wydanego projektu	8

1. Wstęp

a) Tworzenie nazw plików aplikacji

Projekt składa się z dwóch folderów:

- i) ocr – zawierającego backend aplikacji
- ii) Frontend - zawierającego frontend aplikacji

b) Tworzenie nazw plików testowych

Główny folder testowy w naszym środowisku “django” znajduje się w folderze “ocr/application/tests”.

W folderze “tests” znajdują się foldery “unit”, “integration”, “e2e” zawierające odpowiednio testy jednostkowe, testy integracyjne oraz testy sprawdzające działanie systemu w tym części frontendowej. W poszczególnych folderach testy są nazwane w sposób intuicyjny np. “test_contact” sprawdza działanie formularza kontaktowego.

c) Wprowadzenie

Nasza aplikacja jest aplikacją ocr zawierającą w sobie również opcje logowania, rejestracji oraz przeglądania przestanych zdjęć przez użytkownika. Posiada wiele rozbudowanych funkcjonalności, które mają ułatwić użytkowanie i dać wiele możliwości użytkownikowi. Aplikacja powinna być niezawodna i radzić sobie z niechcianym zachowaniem użytkownika.

d) Podejście do testów

Testowanie projektu ma na celu sprawdzenie:

- I) Czy spełnia ona wszystkie postawione na początku wymagania
- II) Czy wszystkie funkcjonalności działają normalnie
- III) Czy projekt jest odporny na niestandardowe działanie użytkownika
- IV) Czy nie wkradły się błędy ludzkie
- V) Czy wszystkie ograniczenia działają
- VI) Czy interfejs użytkownika jest przejrzysty i wszystko jest łatwe do znalezienia
- VII) Czy aplikacja zachowuje się zgodnie z założeniami

2. Opis testów

a) Testowany obiekt

Obiektem testowym jest aplikacja do optycznego rozpoznawania tekstu i możliwości eksportu do pliku napisana w języku python z użyciem technologii takich jak django i react.

b) Narzędzia

Wszystkie testy automatyczne napisane zostały w języku python z użyciem Django TestCase, unittest oraz Selenium. Do testowania głównie zostało użyte wbudowane do Django narzędzie do testowania. Do porównywania czy rezultat jest dobry głównie zostało użyte polecenie Assert. Projekt korzysta z systemu kontroli wersji Git, wszelkie zmiany muszą zostać zatwierdzone przez jednego z programistów, lecz w przyszłości planowane jest dodanie systemu automatycznego testowania z użyciem github actions, które będą się uruchamiać automatycznie przy każdym Push lub Pull Request.

c) Funkcjonalność nietestowana

- I) Połączenie internetowe – w obecnych czasach połączenie internetowe jest niezawodne i w zupełności wystarcza do komunikacji oraz działania naszej aplikacji
- II) Zasoby bazy danych – darmowa wersja bazy danych firebase posiada ograniczenia na ilość, pomimo to do działania naszej aplikacji wystarcza, ponieważ mamy wprowadzone pewne limity
- III) Ilość użytkowników - baza danych radzi sobie bardzo dobrze nawet z dużą liczbą użytkowników, ponieważ pola użytkowników zawierają znikomą część dostępnego miejsca w bazie danych
- IV) Logowanie, Rejestracja – aktualnie nie jest wprowadzona i działa tylko przez google'a. Została przetestowana manualnie, lecz nie ma dla niej napisanych testów.

3. Plan testów

a) Opis

Testowanie jest głównie oparte na elementy wbudowane django takie jak forms, models, services. Duża część testów sprawdza działanie api co jest kluczowe z

naszym projekcie. Ponadto sprawdzamy działanie aplikacji od strony użytkownika i planujemy jakie działania może podjąć. Testy jednostkowe sprawdzają najmniejsze części kodu bez ingerencji z innym elementami. W tym typie testów najczęściej używamy mockowania w celu udawania działania pewnych elementów i symulowania ich działania, jeżeli są potrzebne do działania pewnego komponentu. Testy integracyjne sprawdzają działanie zależne pomiędzy dwoma lub więcej komponentami oraz ich współpracę. Testy e2e głównie sprawdzają działanie aplikacji zaczynając od strony frontendowej, używając również bazy danych i backendu. Testy e2e są głównie zaimplementowane w selenium.

b) Obszar pokrycia testów

- I) Logowanie do systemu
 - Poprawne dane
 - Niepoprawne dane
 - Puste
 - Brak – funkcja gościa
- II) Wylogowanie z systemu
 - Kiedy użytkownik jest zalogowany
 - Kiedy użytkownik nie jest zalogowany
 - Próba wylogowania przy funkcji gościa
 - Poprawne wylogowanie
 - Przełogowanie
- III) Rejestracja do systemu
 - Tworzenie użytkownika
 - Tworzenie użytkownika o danych które już istnieją
- IV) Praca elementu nawigacyjnego strony
 - Zachowanie przy zalogowanym użytkowniku
 - Zachowanie przy niezalogowanym użytkowniku
- V) Działanie formularza kontaktowego
 - Poprawne dane
 - Niepoprawne dane
 - Sprawdzenie ograniczenia długości
 - Przycisk upload formularza
 - Zachowanie pól przy niestandardowych danych
- VI) Działanie formularz uploadu pliku
 - Wybór wszystkich opcji
 - Poprawne dane
 - Niepoprawne dane
 - Upload bez pliku

- Wybór opcji eksportu do pliku
 - Wybór opcji rozmiaru czcionki
 - Wybór opcji czy z confidence scorem
 - Przycisk Upload pliku
 - Zachowanie pól przy niestandardowych danych
- VII) Działanie strony backendowej tj. Zachowania wszystkich serwisów, modeli, widoków itd.
- VIII) Działanie strony administratora – testowanie ręczne przez testerów
- IX) Działanie strony results użytkownika - testowanie ręczne przez testerów

4. Finalny zakres testów

- a) Test_api_services
- b) Test_api_views
- c) Test_forms
- d) Test_manage
- e) Test_models
- f) Test_ocr_model
- g) Test_perform_ocr
- h) Test_services
- i) Test_set_staff
- j) Test_utils
- k) Test_auth
- l) Test_contact
- m) Test_csrf
- n) Test_login_logout
- o) Test_ocr_model
- p) Test_registration
- q) Test_contact
- r) Test_navbar_admin_authorized
- s) Test_navbar_admin_not_authorized
- t) Test_upload
- u) Test_user_profile

5. Zakres pokrycia testami

Testy zostały zaimplementowane zgodnie z planem poza testami na logowanie i rejestrację z powodu braku czasu, działa ona tylko przez autoryzację google'a.

Strona user profile została przetestowana ręcznie przez testera oprogramowania.

Uzyskailiśmy pokrycie testów wynoszące 87%, ponieważ testowanie niektórych funkcji nie miało sensu, bo są trywialne, a niektóre były bardziej opłacalne w testowaniu ręcznym. Plik z pokryciem testów znajduje się w lokalizacji <ocr/htmlcov/index.html>.

File ▲	statements	missing	excluded	coverage
api__init__.py	0	0	0	100%
api\serializers.py	11	0	0	100%
api\services.py	92	26	0	72%
api"urls.py	10	0	0	100%
api\views.py	325	49	0	85%
application__init__.py	0	0	0	100%
application\apps.py	11	2	0	82%
application\forms.py	8	0	0	100%
application\middleware.py	10	4	0	60%
application\model__init__.py	0	0	0	100%
application\model\easyocr.py	11	0	0	100%
application\model\modelbase.py	7	0	0	100%
application\model\paddleocr.py	14	0	0	100%
application\models.py	8	0	0	100%
application\services.py	77	0	0	100%
application\utils.py	15	0	0	100%
manage.py	23	4	2	83%
ocr__init__.py	0	0	0	100%
ocr\settings.py	46	0	0	100%
ocr"urls.py	7	0	0	100%
set_staff.py	50	6	21	88%
Total	725	91	23	87%

6. Raport błędów wykrytych w czasie działania aplikacji

- Przekierowanie na nieistniejącą i nieaktualną stronę logowania na admina – naprawione
- Formularz kontaktowy:
 - Mało estetyczny formularz kontaktowy – przerobione UI – naprawione
 - Brak informacji o przestaniu - naprawione

- III) Brak limitu na pole wiadomości - zostawione
- c) Formularz ocr:
 - I) Brak wysyłania do bazy danych – naprawione
 - II) Zanikanie niektórych zdjęć - zostawione (specyficzne zdjęcie i bardzo rzadko)
- d) Profil użytkownika
 - I) Podwójny pasek nawigacji – naprawione
 - II) Brak ładowania zdjęć z bazy danych – naprawione
- e) Model OCR
 - I) Niepoprawne wyniki dla skomplikowanych zdjęć - w dużej mierze zostawione z pewnymi modyfikacjami

7. Końcowy raport i potwierdzenie zgodności końcowej wydanej aplikacji

Użycie testów automatycznych z użyciem testów manualnych pozwoliły na znalezienie większości błędów oraz sprawdzenia poprawności działania aplikacji. Błędów z logowanie oraz rejestracją bez użycia autoryzacji google'a niestety nie udało się zaimplementować, ale ta z użyciem google'a działa bez zarzutu. Testy automatyczne pokryły 87% kodu, a co za tym idzie aplikacja może posiadać minimalne Bugi, dziwne nieodkryte działania oraz defekty. Całym zespołem testowaliśmy aplikację i nie wykazywała ona niepożądanych zachowań. Użycie testów znacząco zmniejszyło czas odnajdywania błędów oraz analizy zachowań napisanego kodu, a co za tym idzie spójności całej aplikacji. Wcześniej zaplanowana architektura i schematy pozwoliły na proste wprowadzenie testów oraz ich implementację. W głównej mierze chcieliśmy, żeby aplikacji była prosta w obsłudze, wszystkie opcje łatwo dostępne dla użytkownika, a sama aplikacja bez awaryjna. Pomimo nielicznych niedociągnięć aplikacja jest w pełni używalna, odporna na błędy krytyczne oraz skazana na satysfakcję przyszłych użytkowników oraz rozwój opcji i udogodnień.